THỰC HÀNH KIỂM THỬ PHẦN MỀM — TUẦN 06

TỔNG QUAN KIỂM THỬ TỰ ĐỘNG & DEMO ĐĂNG NHẬP

Trình bày vòng 1: Kiến trúc Appium, Tháp kiểm thử & Kịch bản thực thi trên App Mobile

NHÓM THỰC HIỆN

Nhóm Mobile App (Repo: kiemthu_02)

CÔNG NGHỆ CHÍNH

Python 3.14 · pytest · Appium 2 · UiAutomator2

ĐỐI TƯỢNG KIỂM THỬ

Swag Labs Android (Sauce Labs Sample App)

Kiểm Thử Tự Động: Bản Chất, Lợi Ích & Giới Hạn

Hiểu rõ mục đích và cân nhắc kinh tế trước khi tự động hóa

📌 Bản chất & Lợi ích cốt lõi

Kiểm thử tự động (Automation Testing): Sử dụng mã lập trình để điều khiển phần mềm, nạp dữ liệu và kiểm tra kết quả thực tế với mong đợi mà không cần con người thao tác trực tiếp.

- **Tốc độ & Tính lặp lại:** Chạy lại hàng trăm kịch bản kiểm thử hồi quy (Regression Testing) trong vài phút khi có bản build mới.
- **Độ tin cậy cao:** Không mệt mỏi, không bỏ sót bước thao tác, dữ liệu nhập chính xác tuyệt đối.
- **Tiết kiệm chi phí dài hạn:** Chi phí đầu tư ban đầu cao cho việc viết script, nhưng chi phí trên mỗi lần chạy giảm mạnh về 0.
- **Tích hợp CI/CD:** Tự động phát hiện lỗi sớm (Shift Left) ngay khi merge code.

⚠️ Khi nào NÊN và KHÔNG NÊN dùng?

NÊN TỰ ĐỘNG HÓA

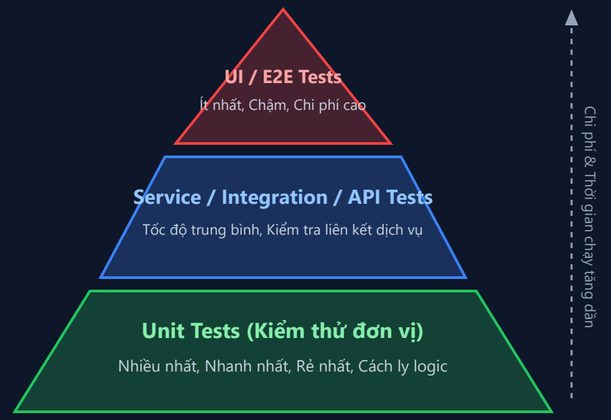
- Chức năng trọng yếu: Đăng nhập, thanh toán, luồng nghiệp vụ cốt lõi (Core Business Flow).
- Kiểm thử hồi quy lặp đi lặp lại qua nhiều phiên bản.
- Kiểm thử dữ liệu lớn (Data-driven) với nhiều bộ tham số.

KHÔNG NÊN DÙNG

- Giao diện/nghiệp vụ đang thay đổi liên tục hàng ngày (Script sẽ liên tục gãy).
- Kiểm thử thăm dò (Exploratory Testing) đòi hỏi tư duy sáng tạo của con người.
- Kiểm thử trải nghiệm người dùng (UX), cảm quan thẩm mỹ đồ họa.
- Tính năng chỉ chạy 1 lần duy nhất (Chi phí viết test lớn hơn chạy tay).

Các Cách Tiếp Cận & Tháp Kiểm Thử (Test Pyramid)

Mô hình phân tầng theo Martin Fowler giúp phân bổ nguồn lực tối ưu



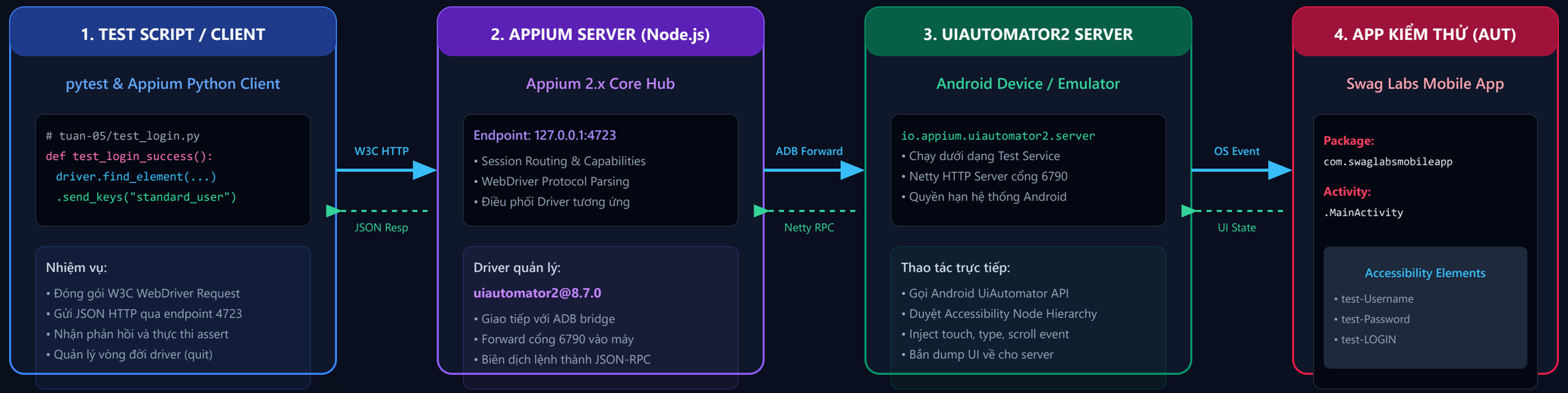
⚖️ Vị trí của dự án trong Tháp Kiểm Thử

Dự án của team tập trung vào tầng **UI / End-to-End Tests** trên thiết bị di động (Mobile App):

- **Đặc thù UI Mobile Test:** Trực tiếp tương tác lên các thành phần đồ họa (nút bấm, ô nhập, thông báo) giống 100% như người dùng thật.
- **Thách thức lớn nhất:** Tốc độ khởi động máy ảo/app chậm, dễ bị "Flaky" (chập chờn) nếu mạng lag hoặc hiệu ứng chuyển trang chưa kết thúc.
- **Giải pháp khắc phục:**
  - Không dùng kiểm thử UI cho mọi trường hợp chi tiết (để Unit test lo).
  - Thiết kế các luồng E2E có giá trị nghiệp vụ cao nhất.
  - Tuyệt đối áp dụng **Explicit Wait (Chờ tường minh)**, không dùng `time.sleep`.

Kiến Trúc Hoạt Động Của Appium 2 & UiAutomator2

Sơ đồ 4 tầng từ mã kiểm thử Python đến đối tượng Swag Labs trên thiết bị Android



Chiến Lược Xác Định Phần Tử (Locator Strategy)

3 việc cốt lõi của kiểm thử giao diện & Bảng xếp hạng độ bền vững của Locator

🎯 3 Việc cốt lõi của kiểm thử UI

1. Tìm phần tử (Find Element): Định vị chính xác tọa độ/đối tượng trên cây UI thông qua Driver.

2. Thao tác (Interact): Nhập văn bản (`send_keys`), xóa dữ liệu cũ (`clear`), chạm/bấm (`click`), vuốt màn hình.

3. Kiểm tra kết quả (Assert): So sánh trạng thái thực tế hiển thị với dữ liệu kỳ vọng bằng `assert`.

🏆 Thứ tự ưu tiên Locator trên Android

Độ ưu tiên	Phương thức	Thuộc tính Android	Đánh giá & Vì sao?
Số 1 (Tốt nhất)	`ACCESSIBILITY_ID`	`content-desc`	Bền nhất. Gắn liền mục đích trợ năng, độc lập bố cục giao diện, chạy được cả iOS/Android.
Số 2 (Khuyên dùng)	`ID`	`resource-id`	Rất tốt khi lập trình viên gán id duy nhất trong layout XML.
Số 3	`-android uiautomator`	`UiSelector()`	Mạnh mẽ khi cần tìm theo văn bản text hoặc quan hệ cha-con nội bộ Android.
Hạn chế tối đa	`XPATH (Tuyệt đối)`	`/hierarchy/...`	Cực kỳ dễ gãy khi đổi layout, hiệu năng phân tích cây DOM mobile rất chậm.

** Mẫu chuẩn đã chạy trong repo: `AppiumBy.ACCESSIBILITY_ID, "test-Username"`*

Kịch Bản Kiểm Thử Đăng Nhập (Tuần 05)

3 Test Case hoàn chỉnh: 1 trường hợp thành công & 2 trường hợp xử lý lỗi biên

TC-05-01: Đăng nhập hợp lệ

Loại: Positive (Thành công)

Dữ liệu:

```
Username: standard_user
Password: secret_sauce
```

Các bước thực hiện:

- Nhập Username và Password
- Bấm nút LOGIN

Kết quả mong đợi:

✓ Chuyển hướng vào màn hình chính, xuất hiện chữ **PRODUCTS**.

TC-05-02: Tài khoản bị khóa

Loại: Negative (Thất bại)

Dữ liệu:

```
Username: locked_out_user
Password: secret_sauce
```

Các bước thực hiện:

- Nhập tài khoản bị khoá
- Bấm nút LOGIN

Kết quả mong đợi:

✓ Hiển thị thông báo: "*Sorry, this user has been locked out.*"

TC-05-03: Bỏ trống Mật khẩu

Loại: Negative (Bỏ trống)

Dữ liệu:

```
Username: standard_user
Password: (bỏ trống)
```

Các bước thực hiện:

- Nhập Username, không nhập Pass
- Bấm nút LOGIN

Kết quả mong đợi:

✓ Hiển thị thông báo: "*Password is required*"

Giải Thích Mã Nguồn & Chiến Lược Chờ Tường Minh

Tránh lỗi flaky bằng WebDriverWait và quản lý vòng đời Driver an toàn

💻 Cấu trúc chuẩn trong `test_login.py`

```
def test_login_success(): driver = None try: # 1. Khởi tạo phiên kết nối Appium Server driver = webdriver.Remote("http://127.0.0.1:4723", options=options) wait = WebDriverWait(driver, 15) # 2. Thao tác: Nhập thông tin & Bấm Login login(driver, "standard_user", "secret_sauce") # 3. Chờ tường minh màn hình kết quả (PRODUCTS) header = wait.until(EC.visibility_of_element_located(( AppiumBy.XPATH, "//*[@text='PRODUCTS']" ))) assert header.is_displayed(), "Không thấy màn hình Products" finally: # 4. BẮT BUỘC: Đóng driver giải phóng session if driver: driver.quit()
```

⏱️ Vì sao KHÔNG dùng `time.sleep` ?

- **Lãng phí thời gian:** `time.sleep(10)` bắt hệ thống đứng yên 10 giây dù phần tử đã xuất hiện sau 1 giây. Hàng trăm test case sẽ làm bộ test chạy mất hàng giờ.
- **Vẫn dễ bị đứt (Flaky):** Nếu mạng hoặc máy giật quá 10 giây, test vẫn fail vô lý.
- **Ưu thế của `WebDriverWait` (Explicit Wait):**
  - Liên tục thăm dò (polling mỗi 500ms).
  - Tiếp tục chạy **ngay khi** phần tử đạt điều kiện (tiết kiệm thời gian tối đa).
  - Chỉ timeout và báo lỗi khi thực sự quá thời gian chờ tối đa.
- **Quản lý phiên trong `finally`:** Đảm bảo phiên Appium luôn được đóng kể cả khi test bị lỗi (assert fail).

Tổng Kết Vòng 1 & Kế Hoạch Các Tuần Tiếp Theo

Hoàn thành giai đoạn làm quen và chuẩn bị cho các kịch bản kiểm thử phức tạp

✅ Đã hoàn thành (Tuần 04 - Tuần 06)

- **Tuần 04:** Cài đặt môi trường Appium 2, UiAutomator2, Android SDK; chạy thành công bài smoke test đầu tiên trên app Swag Labs.
- **Tuần 05:** Viết tài liệu kịch bản `testcases.md`; hoàn thiện 3 bài kiểm thử Đăng nhập (thành công, tài khoản khoá, bỏ trống pass).
- **Tuần 06:** Hệ thống hóa kiến trúc Appium 4 tầng, hiểu sâu về Tháp kiểm thử, hoàn thành bài báo cáo và trình bày vòng 1.

🚀 Định hướng tuần tiếp theo (Tuần 07 - Tuần 11)

- **Tuần 07 (Chức năng 2 & Chờ đúng lúc):** Thí nghiệm so sánh việc có/không có lệnh chờ; cài đặt 2-3 bài test cho Chức năng 2 (ví dụ: Xem chi tiết & Thêm giỏ hàng).
- **Tuần 08 (Chức năng 3 & Báo cáo HTML):** Tích hợp `pytest-html`; chạy toàn bộ test bằng 1 câu lệnh duy nhất xuất `report.html`.
- **Tuần 09 (Ổn định hóa):** Chạy kiểm tra độ ổn định 3 lần liên tiếp không lỗi; quay video demo 3 phút.
- **Tuần 10 - 11:** Trình bày vòng 2 & Nộp báo cáo tổng kết môn học.